Fairness as a controlled-variable strategy

One frozen specification, two implementations, exactly one variable allowed to differ

FROZEN SPECIFICATION

written and frozen before any code

- public API
- UE-context state machine
- data structures and their algorithmic strategy
- sharding granularity and the concurrency model
- the parity contract

the primary control against implementation bias

Also held fixed across both

Allocator
same jemalloc, system build as control

Optimisation
-O2 against opt-level = 2

Hashing
SplitMix64 and FNV-1a, pinned

Workload
one driver, same op sequence per seed

Instrumentation
same HDR config, repeats, interleaving

C++23 implementation

STL, RAII, std::thread

VARIED: implementation language

the study's single independent variable

Rust implementation

stable toolchain, zero unsafe

Equivalence is checked, not assumed

A language-neutral conformance suite runs the same vectors on both sides. The canonical transcripts are then diffed op for op.

Logged, not removed

Where the two languages cannot express the same construct, the divergence goes into the forced-idiom log and is weighed explicitly in the discussion rather than smoothed over.

Acknowledged, not removed

One person wrote both implementations and is not equally expert in both. The parity suite bounds this for behaviour but not for performance, so it is a threat to validity.

Every measured difference is attributable to the language only to the extent that this diagram holds.